Skill _week-4_2520090235

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

Code:

```
 GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t child1, child2;
    int status;

    printf("Session 7 - Process Synchronization\n");
    printf("Parent PID: %d\n\n", getpid());

    child1 = fork();

    if (child1 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0) {
        printf("Child 1 started. PID: %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0) {
        printf("Child 2 started. PID: %d\n", getpid());
        sleep(4);
        printf("Child 2 completed.\n");
        exit(20);
    }

    printf("Parent is monitoring both child processes...\n");

    waitpid(child1, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child 1 (PID %d) exited with status %d.\n",
               child1, WEXITSTATUS(status));
    }

    waitpid(child2, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child 2 (PID %d) exited with status %d.\n",
               child2, WEXITSTATUS(status));
    }

    printf("Parent: All child processes completed.\n");

    return 0;
}
```

Output:

```
Parent PID: 2475

Child 1 started. PID: 2476
Parent is monitoring both child processes...
Child 2 started. PID: 2477
Child 1 completed.
Parent: Child 1 (PID 2476) exited with status 10.
Child 2 completed.
Parent: Child 2 (PID 2477) exited with status 20.
Parent: All child processes completed.
```

Observation:

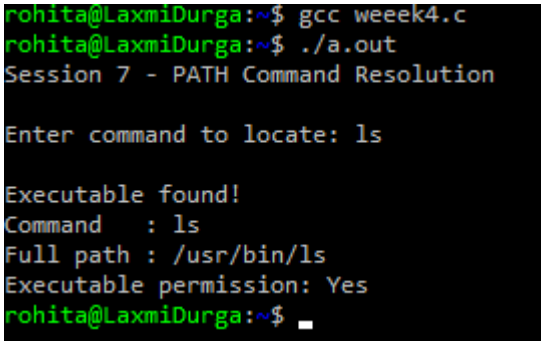
To synchronize and monitor child processes using waitpid(), verify their completion and exit status with WIFEXITED() and WEXITSTATUS(), and ensure proper parent-child process synchronization.

Task2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

Code:

```
 GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATHS 100
#define MAX_LENGTH 4096

int main() {
    char *path_env;
    char *path_copy;
    char *directory;
    char full_path[MAX_LENGTH];
    char command[100];

    printf("Session 7 - PATH Command Resolution\n");

    path_env = getenv("PATH");

    if (path_env == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = malloc(strlen(path_env) + 1);

    if (path_copy == NULL) {
        perror("malloc failed");
        return 1;
    }

    strcpy(path_copy, path_env);

    printf("\nEnter command to locate: ");
    scanf("%99s", command);

    directory = strtok(path_copy, ":");

    while (directory != NULL) {

        snprintf(full_path, sizeof(full_path),
                 "%s/%s", directory, command);

        if (access(full_path, X_OK) == 0) {
            struct stat file_info;

            if (stat(full_path, &file_info) == 0) {
                printf("\nExecutable found!\n");
                printf("Command   : %s\n", command);
                printf("Full path : %s\n", full_path);
                printf("Executable permission: Yes\n");
                free(path_copy);
                return 0;
            }
        }

        directory = strtok(NULL, ":");
    }

    printf("\nCommand not found in PATH.\n");
    printf("Command: %s\n", command);
```

Output:



Observation:

- Retrieval, splitting, and searching of the `PATH` environment variable allows for the location of executable commands across system directories.
- Utilization of the `access()` function verifies executable permissions and ensures effective handling when a command cannot be found.

Session 8:

Task 1:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Code:

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

struct Variable {
    char name[50];
    char value[100];
};

struct Variable vars[MAX];
int count = 0;

/* Find variable */
int findVariable(char name[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(vars[i].name, name) == 0) {
            return i;
        }
    }
    return -1;
}

/* Set or update variable */
void setVariable(char name[], char value[]) {
    int pos = findVariable(name);

    if (pos == -1) {
        strcpy(vars[count].name, name);
        strcpy(vars[count].value, value);
        count++;
    } else {
        strcpy(vars[pos].value, value);
    }
}

/* Expand variables */
void expand(char input[], char output[]) {
    int i = 0, j = 0;

    while (input[i] != '\0') {

        if (input[i] == '$') {

            i++;

            char name[50];
            int k = 0;

            while (input[i] != '\0' &&
                   (input[i] >= 'A' && input[i] <= 'Z' ||
                    input[i] >= 'a' && input[i] <= 'z' ||
                    input[i] >= '0' && input[i] <= '9' ||
                    input[i] == '_')) {

                name[k++] = input[i];
                i++;
            }

            name[k] = '\0';
```

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
                name[k++] = input[i];
                i++;
            }

            name[k] = '\0';

            int pos = findVariable(name);

            if (pos != -1) {
                strcpy(&output[j], vars[pos].value);
                j += strlen(vars[pos].value);
            } else {
                printf("Undefined variable: $%s\n", name);
            }

        } else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

/* Display variables */
void displayVariables() {
    printf("\nVariables:\n");

    for (int i = 0; i < count; i++) {
        printf("%s = %s\n", vars[i].name, vars[i].value);
    }
}

/* Main program */
int main() {

    char input[MAX];
    char name[50];
    char value[100];
    char result[MAX];

    printf("Variable Expansion Program\n");
    printf("--------------------------\n");

    while (1) {

        printf("\nEnter command: ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        /* Exit */
        if (strcmp(input, "exit") == 0) {
            printf("Program terminated.\n");
            break;
        }

        /* Set variable */
        if (strncmp(input, "set ", 4) == 0) {

            sscanf(input + 4, "%s %[^\n]", name, value);

            setVariable(name, value);
```

^G Help ^O Write Out ^F Where Is ^K Cut

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
    for (int i = 0; i < count; i++) {
        printf("%s = %s\n", vars[i].name, vars[i].value);
    }
}

/* Main program */
int main() {

    char input[MAX];
    char name[50];
    char value[100];
    char result[MAX];

    printf("Variable Expansion Program\n");
    printf("--------------------------\n");

    while (1) {

        printf("\nEnter command: ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        /* Exit */
        if (strcmp(input, "exit") == 0) {
            printf("Program terminated.\n");
            break;
        }

        /* Set variable */
        if (strncmp(input, "set ", 4) == 0) {

            sscanf(input + 4, "%s %[^\n]", name, value);

            setVariable(name, value);

            printf("Variable updated: %s = %s\n", name, value
        }

        /* Print / expand */
        else if (strncmp(input, "print ", 6) == 0) {

            expand(input + 6, result);

            printf("Expanded value: %s\n", result);
        }

        /* Display variables */
        else if (strcmp(input, "vars") == 0) {

            displayVariables();
        }

        else {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut

Output:

```
rohita@LaxmiDurga:~$ nano weeekTask4.c
rohita@LaxmiDurga:~$ gcc weeekTask4.c
rohita@LaxmiDurga:~$ ./a.out
Variable Expansion Program
--------------------------

Enter command: set name Rohita
Variable updated: name = Rohita

Enter command: print Hello $name
Expanded value: Hello Rohita
```

```
Enter command: exit
Program terminated.
rohita@LaxmiDurga:~$
```

Observation:

The program successfully detects and expands variable references and handles undefined variables appropriately.
It also supports creating, updating, and displaying variables through simple shell-like commands.

Task 2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Code:

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100
#define MAX_COMMAND 20

/* Function pointer type */
typedef void (*CommandFunction)(char *);

/* Structure for dispatch table */
typedef struct {
    char name[MAX_COMMAND];
    CommandFunction function;
} Command;

/* Shell state */
int running = 1;

/* Built-in command functions */
void builtin_pwd(char *args)
{
    char path[500];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void builtin_cd(char *args)
{
    if (args == NULL || strlen(args) == 0) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args) == 0)
        printf("Directory changed successfully.\n");
    else
        perror("cd");
}

void builtin_echo(char *args)
{
    if (args != NULL)
        printf("%s\n", args);
}

void builtin_help(char *args)
{
    printf("\nAvailable Built-in Commands:\n");
    printf("  pwd          Display current directory\n");
    printf("  cd <dir>     Change directory\n");
    printf("  echo <text>  Display text\n");
    printf("  help         Display available commands\n");
    printf("  exit         Exit the shell\n");
}

void builtin_exit(char *args)
{
```

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
{
    printf("Exiting shell...\n");
    running = 0;
}

/*
 * Dispatch table
 * Each command is associated with its function.
 */
Command commandTable[] = {
    {"pwd",  builtin_pwd},
    {"cd",   builtin_cd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int commandCount = sizeof(commandTable) / sizeof(commandTable[0]);

/* Search for command in dispatch table */
int findCommand(char *command)
{
    int i;

    for (i = 0; i < commandCount; i++) {
        if (strcmp(command, commandTable[i].name) == 0)
            return i;
    }

    return -1;
}

/* Execute built-in command */
void executeCommand(char *command, char *args)
{
    int index;

    index = findCommand(command);

    if (index == -1) {
        printf("Invalid command: %s\n", command);
        return;
    }

    /* Execute command directly in current process */
    commandTable[index].function(args);
}

/* Main shell loop */
int main()
{
    char input[MAX_INPUT];
    char command[MAX_COMMAND];
    char args[MAX_INPUT];

    printf("Simple Built-in Command Shell\n");
    printf("Type 'help' to see available commands.\n");

    while (running) {

        printf("\nshell> ");
        fflush(stdout);
```

^G Help ^O Write Out ^F Where Is ^K Cut
^X Exit ^R Read File ^\ Replace ^U Paste

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
    return -1;
}

/* Execute built-in command */
void executeCommand(char *command, char *args)
{
    int index;

    index = findCommand(command);

    if (index == -1) {
        printf("Invalid command: %s\n", command);
        return;
    }

    /* Execute command directly in current process */
    commandTable[index].function(args);
}

/* Main shell loop */
int main()
{
    char input[MAX_INPUT];
    char command[MAX_COMMAND];
    char args[MAX_INPUT];

    printf("Simple Built-in Command Shell\n");
    printf("Type 'help' to see available commands.\n");

    while (running) {

        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        /*
         * Separate command and arguments.
         */
        command[0] = '\0';
        args[0] = '\0';

        sscanf(input, "%19s %[^\n]", command, args);

        /* Execute command */
        executeCommand(
            command,
            strlen(args) > 0 ? args : NULL
        );
    }

    return 0;
}
```

Output:

```
rohita@LaxmiDurga:~$ nano wTask42.c
rohita@LaxmiDurga:~$ gcc wTask42.c
rohita@LaxmiDurga:~$ ./a.out
Simple Built-in Command Shell
Type 'help' to see available commands.

shell> help

Available Built-in Commands:
  pwd           Display current directory
  cd <dir>      Change directory
  echo <text>   Display text
  help          Display available commands
  exit          Exit the shell

shell> echo

shell> pwd
/home/rohita

shell> exit
Exiting shell...
rohita@LaxmiDurga:~$
```

Observation:

- The program successfully identifies and executes built-in commands using a dispatch table and function pointers.
- It handles invalid commands and maintains shell state, such as the current directory, during execution.